AMBIRE · $WALLET

CODEBASE X-RAY REPORT

X-Ray Report

WALLETToken · contracts/WALLET/WALLET.sol

CONTRACT	SIZE	COMMIT	FRAMEWORK
WALLETToken	85 nSLOC	`63d08560c` (v2)	Hardhat

Includes entry point map & invariant map

26/10/26

AT A GLANCE

Report Overview

WALLETToken (`contracts/WALLET/WALLET.sol`) | 85 nSLOC | 63d08560c (`v2`) | Hardhat | 26/10/26

85	6	5	7
NSLOC IN SCOPE	ENTRY POINTS	ENFORCED GUARDS	INFERRED INVARIANTS

Contents

01	Protocol Overview	
02	Threat & Trust Model	
03	Invariants	
04	Documentation Quality	
05	Test Analysis	
06	Developer & Git History	
—	X-Ray Verdict	
A	Entry Point Map	Appendix
B	Invariant Map	Appendix
C	Architecture Diagram	See Section 01

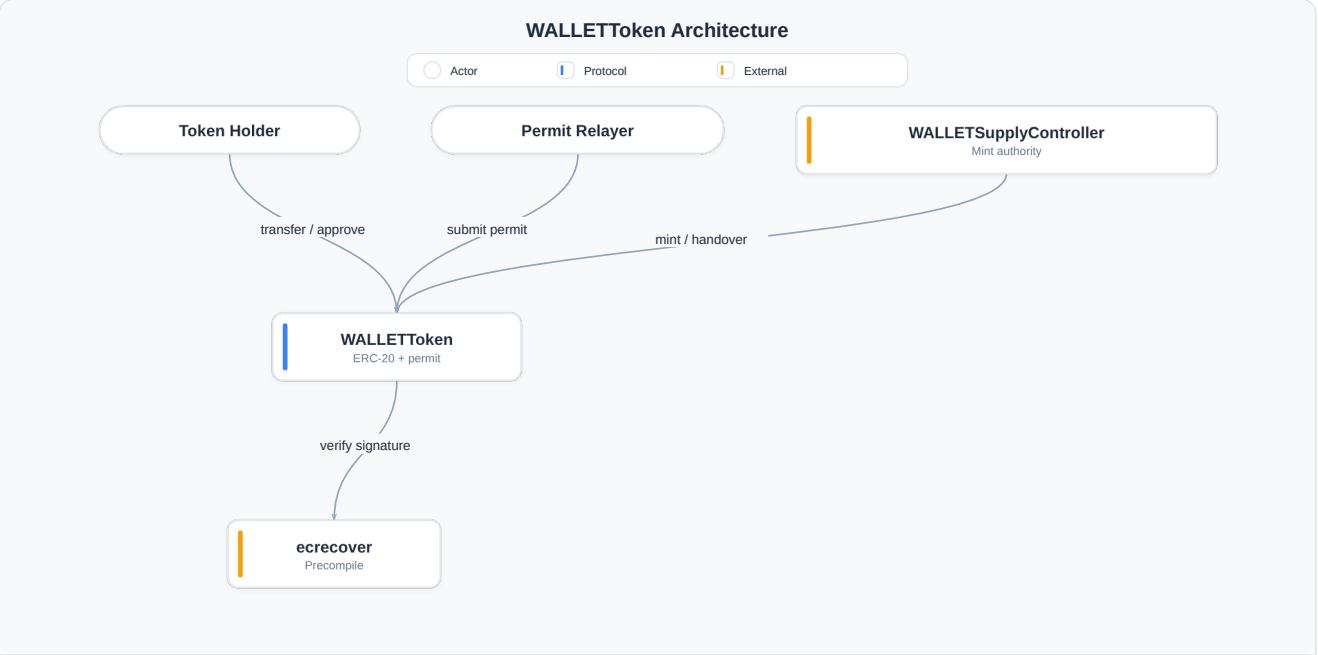
SECTION 01

01 Protocol Overview

What it does: WALLETToken is a minimal ERC-20 with EIP-2612 `permit`. One `supplyController` address can mint up to a hard cap.

- **Users**: Token holders, approved spenders, and relayers that submit signed permits.
- **Core flow**: Holders move tokens with `transfer` / `approve` + `transferFrom`, or grant allowances by signature with `permit`.
- **Key mechanism**: Plain balance mappings with 0.8.19 checked arithmetic. A cached EIP-712 domain separator. A strict `totalSupply < MAX_SUPPLY` cap on mint.
- **Token model**: One token, "Ambire Wallet" (WALLET), 18 decimals, cap 1,000,000,000e18. There is no burn function.
- **Admin model**: A single `supplyController` address can `mint` and `changeSupplyController`. The handover takes one step, with no delay or zero check. In this repo the role holder is `WALLETSupplyController` (governance-gated, out of scope).

For a visual overview of the protocol's architecture, see the architecture diagram.



Architecture diagram — `architecture.svg`

Contracts in Scope

SUBSYSTEM	KEY CONTRACTS	NSLOC	ROLE
Token	WALLETToken	85	ERC-20 balances, EIP-2612 permit, capped mint gated by `supplyController`

How It Fits Together

The core trick: there isn't one. Every state change is a direct mapping update inside one contract, with no external calls apart from the `ecrecover` precompile.

Mint

```
WALLETSupplyController ──► WALLETToken.mint(owner, amount)          WALLET.sol:100
                           ├─ require(msg.sender == supplyController)   L101
                           └─ innerMint()
                               ├─ totalSupply += amount             L93   *state change first*
                               ├─ require(totalSupply < MAX_SUPPLY)  L94  *cap checked after add, strict <*
                               └─ balances[owner] += amount         L95   *owner may be address(0)*
```

Permit then spend

```
Relayer ──► WALLETToken.permit(owner, spender, amount, deadline, v, r, s)   L78
            ├─ require(deadline >= block.timestamp)                   L79
            ├─ digest = keccak(0x1901, DOMAIN_SEPARATOR, ...nonces[owner]++)  L80-84  *nonce consumed inside digest*
            ├─ ecrecover(digest, v, r, s) == owner, != 0             L85-86 *raw ecrecover, ECDSA only*
            └─ allowed[owner][spender] = amount                      L87
Spender ──► WALLETToken.transferFrom(owner, to, amount)               L59
            ├─ balances[from] -= amount                              L60
            ├─ allowed[from][msg.sender] -= amount                   L61   *max allowance also decremented*
            └─ balances[to] += amount                                L62
```

Controller handover

```
supplyController ──► WALLETToken.changeSupplyController(new)         L105
                     ├─ require(msg.sender == supplyController)        L106
                     ├─ emit SupplyControllerChanged                   L108
                     └─ supplyController = new                         L109  *one step, any address incl. 0*
```

SECTION 02

02 Threat & Trust Model

Protocol Threat Profile

> Protocol classified as: **ERC-20 token (no matching DeFi profile)** with **Stablecoin-style mint-authority** characteristics

The code has no swap, lending, vault, staking, bridge or governance signals. The only protocol-shaped feature is capped minting by one controller, which shares the mint-authority concerns of the stablecoin profile, but with no collateral or oracle.

Actors & Adversary Model

ACTOR	TRUST LEVEL	CAPABILITIES
supplyController	Trusted	Instant `mint` to any address up to the cap. Instant one-step `changeSupplyController` to any address, including `address(0)`. No timelock or pause inside the token.
Token holder	Bounded (own balance only)	`transfer`, `approve`, and signing permits for its own balance.
Approved spender	Bounded (up to allowance)	`transferFrom` up to `allowed[from][spender]`.
Permit relayer	Bounded (needs a valid owner signature)	Submits any valid, unexpired permit signature, from the mempool or elsewhere.

Adversary Ranking (ordered by threat level for this protocol type, adjusted by git evidence):

1. **Compromised or misconfigured supply controller** — the one address that can create supply or move the mint authority.
2. **Signature front-runner / replayer** — can submit a holder's permit signature first, or reuse it on another chain that shares the deployment state.
3. **Integrator assumption mismatch** — a caller contract that expects standard behaviour where this token differs: no infinite allowance, ECDSA-only permit, supply counted at `address(0)`.

See entry-points.md for the full permissionless entry point map.

Trust Boundaries

- **Mint authority** — guarded only by `msg.sender == supplyController` (`WALLET.sol:101,106`). There is no delay, cap per call, or pause in the token, and the most powerful single action is the one-step handover at L109.
- **Permit signature** — the only authentication for allowance changes made by others (`WALLET.sol:85-86`). It is bound to the constructor-time `DOMAIN_SEPARATOR` and the per-owner nonce.

Key Attack Surfaces

- **Supply cap comparison** [G-3, I-2, E-1] — `innerMint:93-94` adds before checking, and uses a strict `<` against `MAX_SUPPLY`. Worth confirming the intended maximum, and how callers that mint whole accrued amounts behave near the cap.
- **Supply controller handover** [G-5, I-6, I-7, E-2] — `changeSupplyController:105-110` is one step, with no zero or code-size check. Worth tracing which contracts read `supplyController()` and call into it, and what the governance path to this function is.
- **Permit domain and signature handling** [G-1, G-2, I-4, I-5] — `DOMAIN_SEPARATOR` is cached at L37, and `permit:85` uses raw `ecrecover` with no `s` / `v` range check and no EIP-1271 path. Worth checking fork behaviour, and smart-account owners such as Ambire accounts.
- **Allowance semantics** [I-3] — `transferFrom:61` decrements every allowance, including `type(uint).max`. Worth checking integrators that approve once and expect the approval to be infinite.
- **Supply held at `address(0)`** [I-1] — `mint` and `transfer` accept `address(0)` as recipient, and no burn exists, so such tokens stay in `totalSupply`. Worth confirming how callers that "burn" via `mint(address(0), x)` expect cap headroom and reported supply to behave.

Protocol-Type Concerns

As an ERC-20 token: - `WALLET.sol` makes no external calls and has no transfer hooks, so no reentrancy path exists inside the token. Callers' own call ordering is their concern. - `transfer:52` and `transferFrom:59` have no zero-address or self-address recipient checks. This is standard but non-defensive behaviour, and worth noting for integrators.

Temporal Risk Profile

Deployment & Initialization: - The constructor (`WALLET.sol:28-30`) accepts any `_supplyController`, including `address(0)`. That would make the token unmintable from block one. Not mitigated in code. - `DOMAIN_SEPARATOR` (`WALLET.sol:33-45`) is computed once from the deploy-time `chainid()`. It is correct on the deployment chain, but never recomputed afterwards.

Composability & Dependency Risks

Dependency Risk Map:

> **ecrecover precompile** — via `WALLETToken.permit` - Assumes: returns the signer for a valid `(v, r, s)` and `address(0)` on failure - Validates: rejects `address(0)`; no `s`-range or `v ∈ {27,28}` check (replay still blocked by the nonce) - Mutability: Immutable (EVM precompile) - On failure: reverts `INVALID_SIGNATURE`

Token Assumptions *(unvalidated only)*: - Not applicable. The token holds no other tokens.

Shared State Exposure: - In-repo callers that depend on this token, all out of scope: `WALLETSupplyController` holds `supplyController` and calls `mint`, including `mint(address(0), toBurn)` for penalties. `StakingPool` (`xWALLET.sol`) reads `WALLET.supplyController()` and pulls tokens with `transferFrom`. Any change to the cap, the controller or the allowance behaviour reaches them directly.

SECTION 03

03 Invariants

Full invariant map: invariants.md

A dedicated reference file contains the complete invariant analysis — do not look here for the catalog.

- **5 Enforced Guards** (`G-1` … `G-5`) — per-call preconditions with `Check` / `Location` / `Purpose`
- **7 Single-Contract Invariants** (`I-1` … `I-7`) — Conservation, Bound, StateMachine
- **0 Cross-Contract Invariants** — the scope is a single contract with no external calls
- **2 Economic Invariants** (`E-1` … `E-2`) — higher-order properties deriving from `I-N`

Every inferred block cites a concrete Δ-pair, guard-lift + write-sites, or state edge. All 7 inferred invariants are On-chain=**Yes**. I-2 and I-7 record properties whose exact shape (strict cap, absorbing zero controller) is worth confirming against intent. Attack-surface bullets above cross-link directly into the relevant blocks.

SECTION 04

04 Documentation Quality

ASPECT	STATUS	NOTES
README	Present	`README.md` (repo-level, about the TypeScript library; nothing on the WALLET token)
NatSpec	~0 annotations	No `@notice` / `@dev` / `@param` in `WALLET.sol`
Spec/Whitepaper	Missing	No token spec in the repo
Inline Comments	Sparse	Section markers (`// EIP 2612` , `// Supply control`), the PERMIT_TYPEHASH preimage at L23, and a CEI note at L107

SECTION 05

05 Test Analysis

METRIC	VALUE	SOURCE
Test files	48 (repo-wide, under `test/`)	File scan (always reliable)
Test functions	58 (repo-wide `it(` / `test(` under `test/`)	File scan (always reliable)
Line coverage	Unavailable — no coverage plugin (`solidity-coverage`) configured in `hardhat.config.ts` / `package.json`	Coverage tool (requires compilation)
Branch coverage	Unavailable — same reason	Coverage tool (requires compilation)

`enumerate.sh` uses `grep -P`, which macOS BSD grep does not support. nSLOC and test-function counts were recomputed with `grep -E`.

Test Depth

CATEGORY	COUNT	CONTRACTS COVERED
Unit	0 for WALLETToken (58 repo-wide)	none — no test deploys `WALLETToken`
Fork	2 (skipped)	`test/stkWallet/stkWallet.ts` is `describe.skip` and uses the mainnet WALLET at `0x8880…94ae`, not this source
Stateless Fuzz	0	none
Stateful Fuzz (Foundry/Echidna/Medusa)	0	none
Formal Verification (Certora/Halmos/HEVM)	0	none

Gaps

- No unit test compiles and exercises `WALLETToken` itself: mint cap boundary, permit, allowance and handover.
- No stateful fuzzing for I-1 (supply conservation) or I-2 (cap). These are cheap invariants for a token this size.
- The only WALLET-related suite is skipped and targets the deployed mainnet token, so it does not cover this source file.

SECTION 06

06 Developer & Git History

> Repo shape: squashed_import (for this file) — `WALLET.sol` arrived whole in `54aa2b3e1` (2025-04-23) and has one follow-up change. No development history of the token is visible in this repo.

Analyzed branch: `v2` at `63d08560c`.

Contributors

AUTHOR	COMMITS	SOURCE LINES (+/-)	% OF SOURCE CHANGES
Ivo Georgiev	1	+111 / -0	99%
Borislav Itskov	1	+1 / -1	1%

Review & Process Signals

SIGNAL	VALUE	ASSESSMENT
Unique contributors	2 (file) / 23 (repo)	Small team for this file
Merge commits	0 of 2 file commits (repo: 3,825 of 12,958)	Repo uses merges; neither file commit is a merge
Repo age	2022-05-11 → 2026-04-28	~4 years (repo); file present since 2025-04-23
Recent source activity (30d)	0 commits	Quiet
Test co-change rate	0%	Neither file commit touched test files (measures co-modification, NOT coverage)

File Hotspots

FILE	MODIFICATIONS	NOTE
contracts/WALLET/WALLET.sol	2	Import + one-line pragma change (`ac5bde003`: compiler pinned to 0.8.19 "so hardhat could compile")

Security-Relevant Commits

No development history for this file — fix detection not applicable. The analyzer's repo-wide fix candidates do not touch `WALLET.sol`.

Security Observations

- **Imported, not developed here** — 111 of 111 lines arrived in one commit, `54aa2b3e1`.
- **Compiler changed after import** — `ac5bde003` set `pragma solidity 0.8.19` so Hardhat could compile it. Worth confirming that this matches the compiler used for the deployed bytecode.
- **No test co-changes** — neither commit touched test files.

Cross-Reference Synthesis

- **No tests + import-only history → the source here is unverified against deployment** — confirm that `WALLET.sol` matches the verified mainnet bytecode before relying on any of Sections 2-3.
- **Pragma change (`ac5bde003`) + strict cap at L94 (I-2)** — the behaviour is unchanged by the pragma, but the checked-arithmetic assumptions in I-1 and I-3 depend on the compiler being ≥ 0.8.

SUMMARY

X-Ray Verdict

EXPOSED — the file has no NatSpec and no spec (Docs = EXPOSED), and no test exercises `WALLETToken` directly.

Structural facts: 1. 85 nSLOC in 1 contract, with 6 entry points (4 permissionless, 1 role-gated, 1 admin-only) and no external calls apart from `ecrecover`. 2. One privileged role (`supplyController`), with a one-step handover and no timelock inside the token. 3. 0 NatSpec annotations, and no token spec in the repo. 4. 0 test functions deploy `WALLETToken`. The only WALLET-related suite (2 tests) is skipped and targets the mainnet deployment. 5. The file arrived in 1 import commit (`54aa2b3e1`) plus 1 pragma change (`ac5bde003`). It has 0 merge-reviewed commits.

APPENDIX A

A Entry Point Map

WALLETToken (`contracts/WALLET/WALLET.sol`) | 6 entry points | 4 permissionless | 1 role-gated | 1 admin-only

Protocol Flow Paths

Setup (Deployer)

`constructor(_supplyController)` → sets `supplyController` (L29) + `DOMAIN_SEPARATOR` (L37) ◀— no zero-address check on `_supplyController`

Supply (supplyController)

`[setup above]` → `mint(owner, amount)` → `innerMint()` ◀— `totalSupply + amount < MAX_SUPPLY`

Holder Flow

`[mint above]` → `transfer(to, amount)` ◀— `balances[msg.sender] ≥ amount`

`[mint above]` → `approve(spender, amount)` → `transferFrom(from, to, amount)` ◀— `allowed[from][msg.sender] ≥ amount` and `balances[from] ≥ amount`

`[mint above]` → [owner signs EIP-712 Permit off-chain] → `permit(...)` → `transferFrom(...)` ◀— `deadline ≥ block.timestamp`, signature matches `nonces[owner]`

Admin (supplyController)

`[setup above]` → `changeSupplyController(new)` ◀— `msg.sender == supplyController`; a new value of `address(0)` disables both supplyController functions permanently

Permissionless

`WALLETToken.transfer()`

ASPECT	DETAIL
Visibility	external
Caller	Any token holder
Parameters	to (user-controlled), amount (user-controlled)
Call chain	`→ WALLETToken.transfer()` (no external calls)
State modified	`balances[msg.sender]`, `balances[to]`
Value flow	Tokens: msg.sender → to (any address, including `address(0)` and the token contract)
Reentrancy guard	no (no external calls)

`WALLETToken.transferFrom()`

ASPECT	DETAIL
Visibility	external
Caller	Any address holding an allowance from `from`
Parameters	from (user-controlled), to (user-controlled), amount (user-controlled)
Call chain	`→ WALLETToken.transferFrom()` (no external calls)
State modified	`balances[from]`, `allowed[from][msg.sender]`, `balances[to]`
Value flow	Tokens: from → to
Reentrancy guard	no (no external calls)

`WALLETToken.approve()`

ASPECT	DETAIL
Visibility	external
Caller	Any token holder
Parameters	spender (user-controlled), amount (user-controlled)
Call chain	`→ WALLETToken.approve()`
State modified	`allowed[msg.sender][spender]` (absolute set)
Value flow	None
Reentrancy guard	no

`WALLETToken.permit()`

ASPECT	DETAIL
Visibility	external
Caller	Anyone (relayer or owner); authority comes from the signature
Parameters	owner (user-signed), spender (user-signed), amount (user-signed), deadline (user-signed), v/r/s (user-signed)
Call chain	`→ WALLETToken.permit() → ecrecover` precompile
State modified	`nonces[owner]` (+1), `allowed[owner][spender]` (absolute set)
Value flow	None
Reentrancy guard	no

Role-Gated

`supplyController`

`WALLETToken.mint()`

ASPECT	DETAIL
Visibility	external, `require(msg.sender == supplyController)` (L101)
Caller	Supply controller (deployed controller is `WALLETSupplyController`, out of scope)
Parameters	owner (keeper-provided), amount (keeper-provided)
Call chain	`→ WALLETToken.mint() → WALLETToken.innerMint()`
State modified	`totalSupply`, `balances[owner]`
Value flow	New supply → owner (any address, including `address(0)`)
Reentrancy guard	no (no external calls)

Admin-Only

CONTRACT	FUNCTION	PARAMETERS	STATE MODIFIED
WALLETToken	`changeSupplyController()`	newSupplyController (any address, no zero check)	`supplyController` (one step, no delay)

Initialization

CONTRACT	FUNCTION	PARAMETERS	STATE MODIFIED
WALLETToken	`constructor()`	_supplyController (no zero check)	`supplyController`, `DOMAIN_SEPARATOR`

APPENDIX B

B Invariant Map

WALLETToken (`contracts/WALLET/WALLET.sol`) | 5 guards | 7 inferred | 0 not enforced on-chain

1. Enforced Guards (Reference)

Per-call preconditions. Heading IDs below (`G-N`) are anchor targets from x-ray.md attack surfaces.

G-1

`require(deadline >= block.timestamp, "DEADLINE_EXPIRED")` · `WALLET.sol:79` · Bounds the lifetime of an off-chain permit signature so a leaked or unused signature cannot be submitted indefinitely.

G-2

`require(recoveredAddress != address(0) && recoveredAddress == owner, "INVALID_SIGNATURE")` · `WALLET.sol:86` · The only authentication on `permit` ; binds the allowance write to the owner's signature over `DOMAIN_SEPARATOR` + `nonces[owner]` , and blocks the `ecrecover` zero-address failure value from impersonating `owner = address(0)` .

G-3

`require(totalSupply < MAX_SUPPLY, 'MAX_SUPPLY')` · `WALLET.sol:94` · Enforces the hard supply cap on the only path that creates tokens; evaluated after the increment at L93.

G-4

`require(msg.sender == supplyController, 'NOT_SUPPLYCONTROLLER')` · `WALLET.sol:101` · Restricts token creation to the single privileged supply-controller address.

G-5

`require(msg.sender == supplyController, 'NOT_SUPPLYCONTROLLER')` · `WALLET.sol:106` · Restricts reassignment of the mint authority to the current holder of that authority.

2. Inferred Invariants (Single-Contract)

Inferred invariants are derived from structural analysis of the source code. Each block below cites one of five extraction methods in its `Derivation` field:

- **Δ-pair (delta-pair) analysis** — two or more storage variables in the same function body that change by equal-and-opposite amounts, implying a conservation law like `A == Σ B[key]` or `A + B = const` .

- **Guard lift** — a `require` / `if-revert` on a storage variable, promoted from a per-call precondition to a global property by checking that *every* other write site of that variable enforces an equivalent guard. If any write site lacks it, the lifted invariant is On-chain=**No** (and a candidate bug).
- **State-machine edge** — a storage variable that transitions through discrete values with no reverse path.
- **Temporal predicate** — a check tied to `block.timestamp`, `block.number`, or a stored duration/deadline variable.
- **NatSpec-stated global property** — a developer-asserted invariant in a NatSpec `@invariant` tag or inline comment. (None present in this file.)

Each block is classified into one of five **categories** by shape: `Conservation` · `Bound` · `Ratio` · `StateMachine` · `Temporal`. Category definitions at the end of §2.

I-1

`Conservation` · On-chain: **Yes**

> `totalSupply == Σ balances[a]` over all addresses, including `balances[address(0)]`.

Derivation — Δ-pair: `WALLET.sol:93 ↔ WALLET.sol:95` (`innerMint`: `Δ(totalSupply) = +amount`, `Δ(balances[owner]) = +amount`). Self-conserving transfers: `WALLET.sol:53 ↔ WALLET.sol:54` (`transfer`) and `WALLET.sol:60 ↔ WALLET.sol:62` (`transferFrom`), each `-amount` / `+amount` under 0.8.19 checked arithmetic. These three functions are the only writers of `balances` and `totalSupply`; no burn path exists, so nothing ever lowers `totalSupply`.

If violated — reported supply diverges from spendable balances; holders could spend tokens that were never minted.

I-2

`Bound` · On-chain: **Yes**

> `totalSupply ≤ MAX_SUPPLY - 1` (i.e. `totalSupply < 1_000_000_000e18`) at all times.

Derivation — guard-lift: `require(totalSupply < MAX_SUPPLY, 'MAX_SUPPLY')` (`WALLET.sol:94`) + write-site enumeration of `totalSupply`: the only write is `WALLET.sol:93` inside `innerMint`, immediately followed by the guard. The strict `<` means the constant `MAX_SUPPLY` itself is never a reachable value.

If violated — supply above the declared cap.

I-3

`Conservation` · On-chain: **Yes**

> In `transferFrom`, the allowance consumed equals the balance debited from `from`: `Δ(allowed[from][msg.sender]) == Δ(balances[from]) == -amount`.

Derivation — Δ-pair: `WALLET.sol:60 ↔ WALLET.sol:61`. `allowed` is otherwise written only by absolute assignment in `approve` (`WALLET.sol:68`) and `permit` (`WALLET.sol:87`). There is no infinite-allowance branch, so `type(uint).max` is decremented like any other value.

If violated — a spender could move more than they were approved for.

I-4

`StateMachine` · On-chain: **Yes**

> `nonces[owner]` is strictly increasing; each value is consumed by at most one successful `permit`.

Derivation — edge: `nonces[owner]@n → n+1` at `WALLET.sol:83` (post-increment inside the signed struct). This is the only write site, and no path decrements or resets it.

If violated — a permit signature could be replayed on the same chain.

I-5

`StateMachine` · On-chain: **Yes**

> `DOMAIN_SEPARATOR` is written exactly once, in the constructor, and embeds the `chainid()` and `address(this)` observed at deployment.

Derivation — edge: `unset → set@WALLET.sol:37` (constructor). This is the only write site, and no function recomputes it, so the stored value does not track `block.chainid` after deployment.

If violated — permit signatures would be bound to a different domain than the one signers expect.

I-6

`Bound` · On-chain: **Yes**

> After construction, only the current `supplyController` can change `supplyController`, and only `supplyController` can create tokens.

Derivation — guard-lift: `require(msg.sender == supplyController, 'NOT_SUPPLYCONTROLLER')` (`WALLET.sol:101`, `WALLET.sol:106`) + write-site enumeration of `supplyController`: `WALLET.sol:29` (constructor) and `WALLET.sol:109` (`changeSupplyController`, guarded by L106). The token-creation path `innerMint` is reached only from `mint` (guarded by L101).

If violated — any caller could mint or take the mint authority.

I-7

`StateMachine` · On-chain: **Yes**

> `supplyController == address(0)` is an absorbing state: once reached, `mint` and `changeSupplyController` are uncallable forever.

Derivation — edge: `supplyController@X → address(0)@WALLET.sol:109`, reachable because `changeSupplyController` has no zero-address check (`WALLET.sol:105-110`) and the constructor does not check `_supplyController` (`WALLET.sol:28-29`). There is no reverse path, because `msg.sender` can never equal `address(0)` at L101/L106.

If violated — n/a (this records a reachable one-way transition, not a protected property).

Categories: - **Conservation**: Two or more storage variables change by equal-and-opposite amounts in the same function body. - **Bound**: A guard on a storage variable, lifted to a global property and enforced across every write site of that variable. - **Ratio**: A storage variable is defined as a formula of other storage variables. (None in this file.) - **StateMachine**: A storage variable transitions through discrete values with guards preventing reversal. - **Temporal**: A condition depends on `block.timestamp` , `block.number` , or a duration/deadline variable stored on-chain. (The only temporal check, G-1, compares a parameter, not storage, so it is not promoted.)

3. Inferred Invariants (Cross-Contract)

None. The scope is a single contract that makes no external calls (only the `ecrecover` precompile). Its callers (`WALLETSupplyController` , `StakingPool` in `xWALLET.sol`) are outside the scope files, so no caller/callee pair can be cited on both sides.

4. Economic Invariants

E-1

On-chain: **Yes**

> The total of all balances, including tokens held at `address(0)` , never exceeds `MAX_SUPPLY - 1` wei.

Follows from — `I-1` + `I-2`

If violated — holders could hold more tokens in total than the hard cap allows.

E-2

On-chain: **Yes**

> New supply can come only from one address at a time, and that authority can be permanently retired by assigning it to `address(0)`.

Follows from — `I-6` + `I-7`

If violated — the minting authority could be shared, or revived after retirement.